Intro to STAC

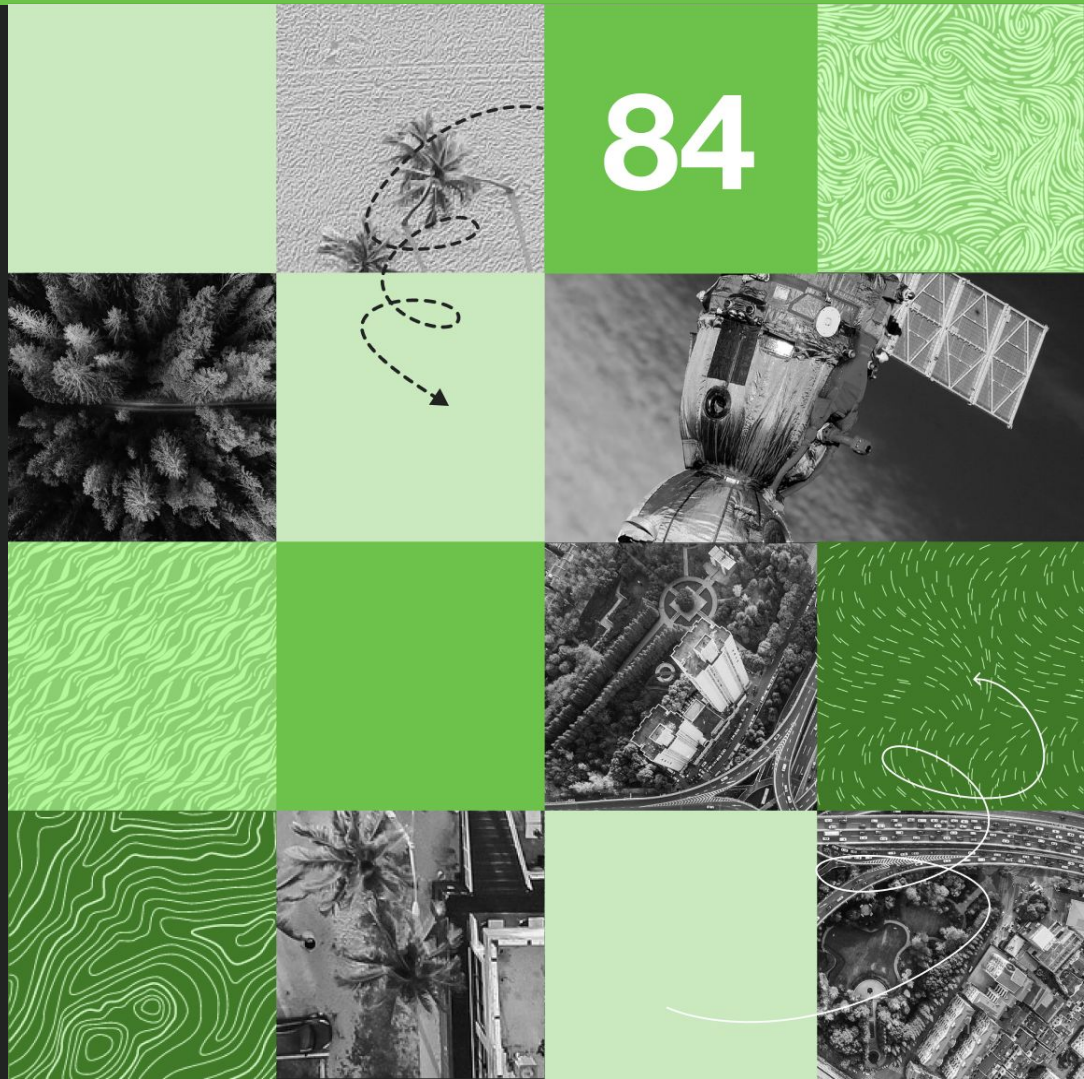
The basics, for users and providers

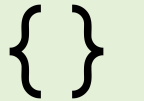
FOSS4G-NA 2025 | Reston, VA
FOSS4G 2025 | Auckland, NZ



Matthew Hanson
@geoskeptic.bsky.social

Element 84





stac-spec

Metadata language for describing geospatial data

Extensions



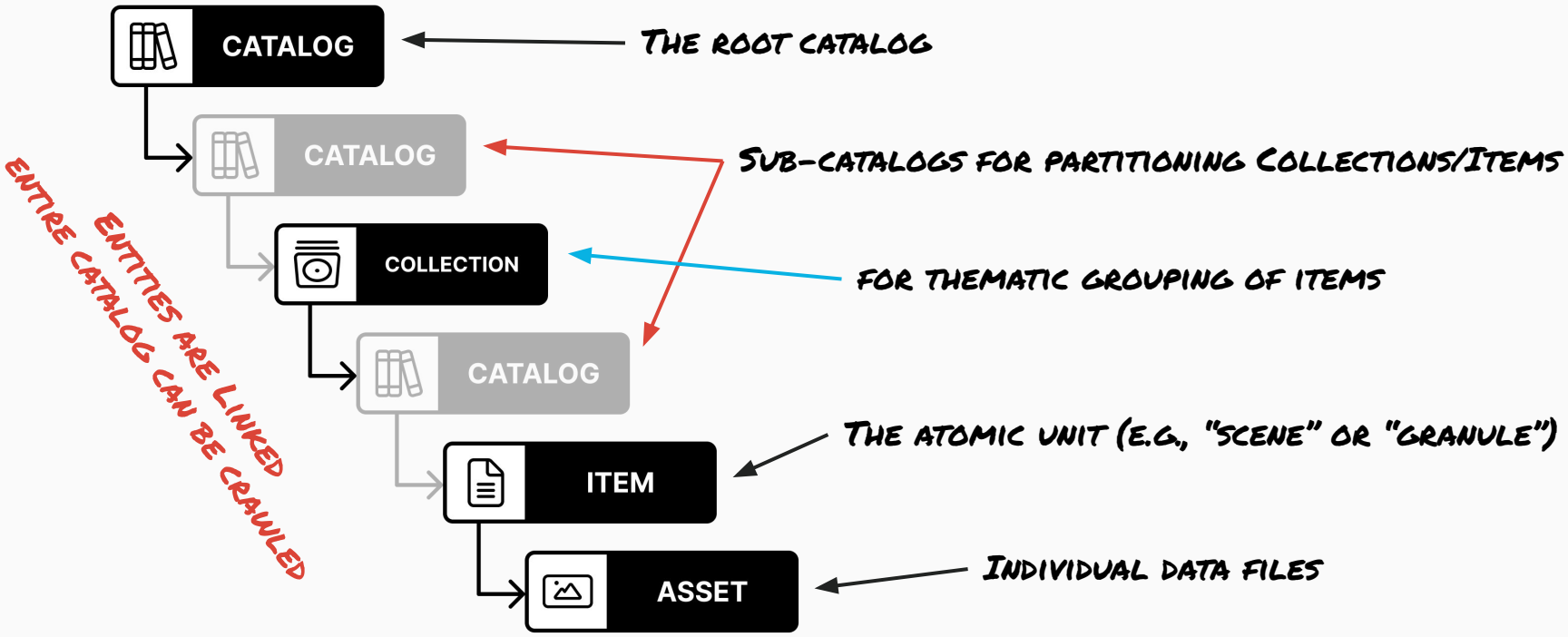


stac-api-spec

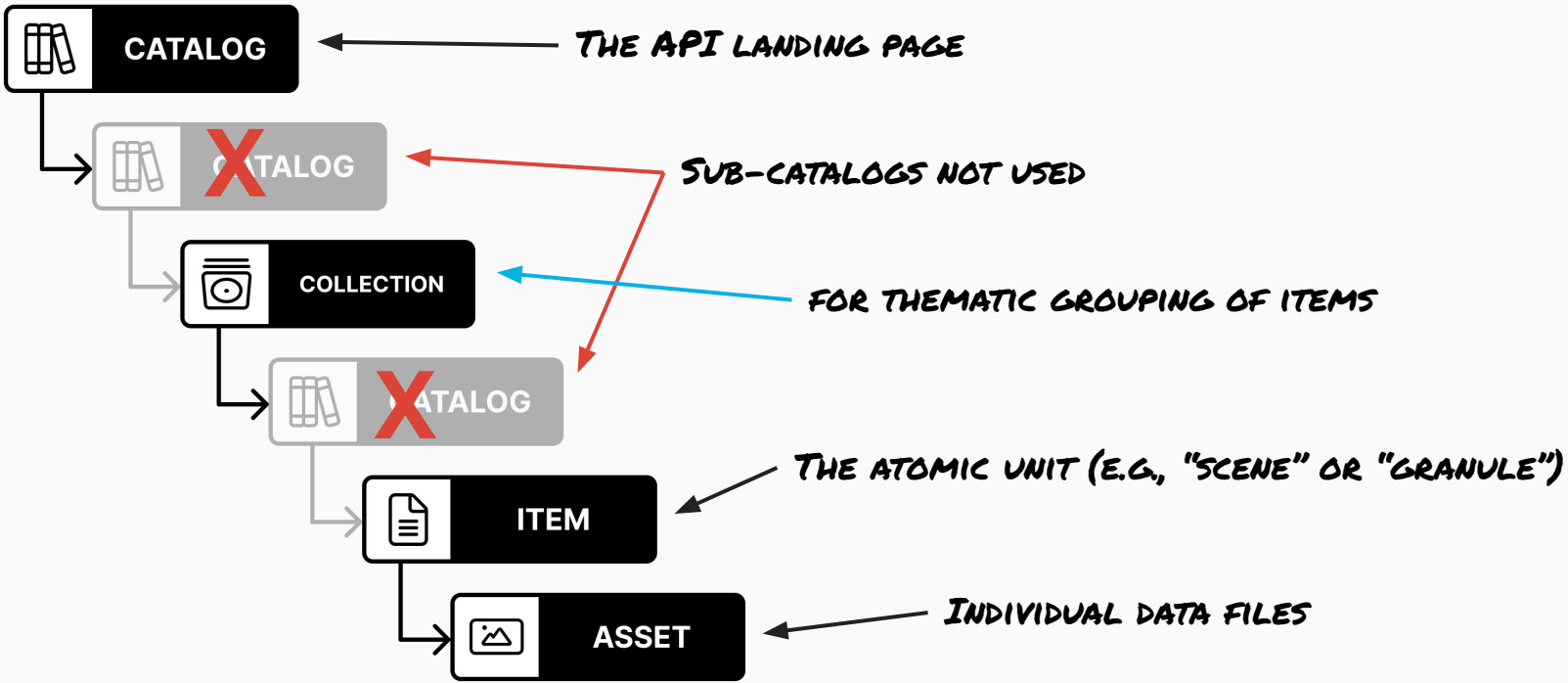
Searching and accessing metadata through an API

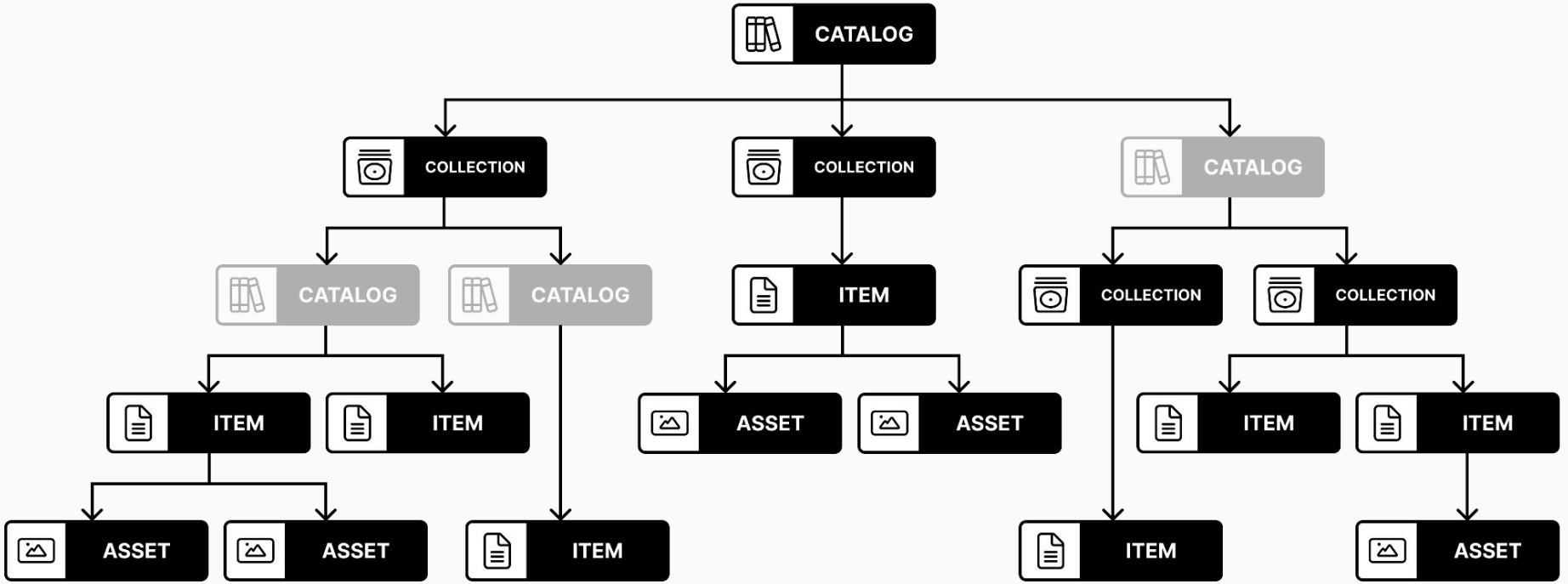
Extensions

Static Catalogs



“Dynamic” Catalog (an API)



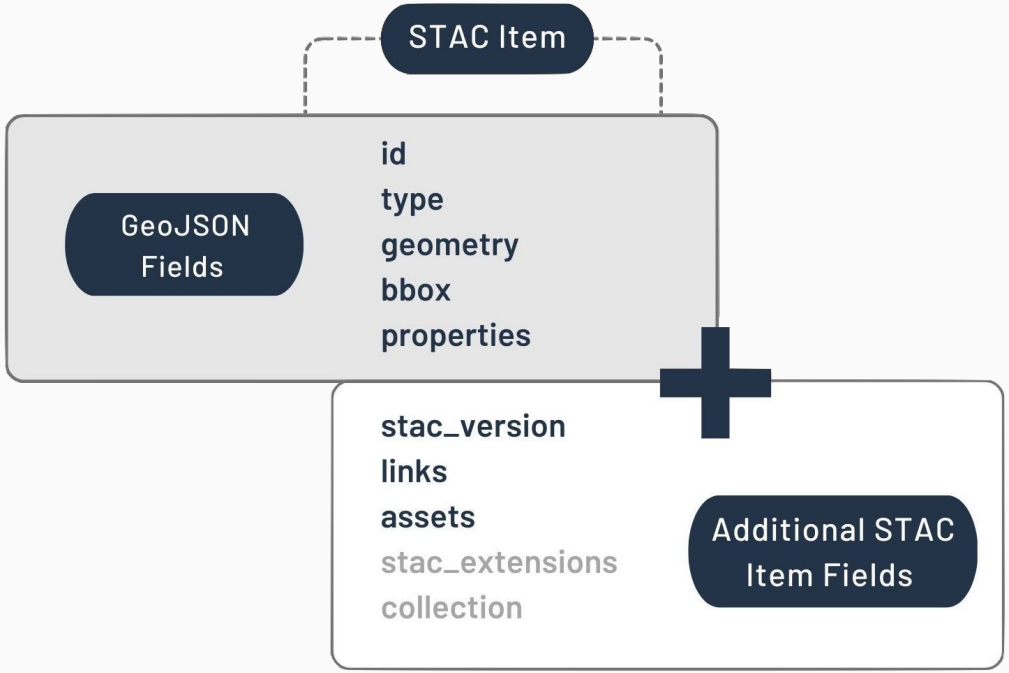

CATALOG
COLLECTION
COLLECTION
CATALOG
CATALOG
CATALOG
ITEM
COLLECTION
COLLECTION
ITEM
ITEM
ASSET
ASSET
ITEM
ITEM
ASSET
ASSET
ITEM
ITEM
ASSET

Key Points

- Built on standards (JSON, GeoJSON, OGC APIs)
- Simple and stable core structure
- Metadata decoupled from data
- Supported by an Open-Source ecosystem

STAC for Users

Anatomy of an Item




```
{
  "type": "Feature",
  "stac_version": "1.0.0",
  "id": "S2A_T22JDM_20240508T132235_L2A",
  "properties": {…
  },
  "geometry": {
    "type": "Polygon",
    "coordinates": [
      [
        [-51.64997287741106, -29.82887736130735],
        [-51.93692244077125, -30.817923786828445],
        [-50.897965022919415, -30.821271019233876],
        [-50.89898692183221, -29.83043772553916],
        [-51.64997287741106, -29.82887736130735]
      ]
    ]
  },
  "links": […
  ],
  "assets": {…
  },
  "bbox": [-51.936922, -30.821271, -50.897965, -29.828877],
  "stac_extensions": […
  ],
  "collection": "sentinel-2-c1-l2a"
}
```

Links


```
"links": [
  {
    "rel": "self",
    "type": "application/geo+json",
    "href": "https://earth-search.aws.element84.com/v1/collections/sentinel-2-c1-l2a/items/S2A_T22JDM_20240508T132235_L2A"
  },
  {
    "rel": "canonical",
    "href": "s3://e84-earth-search-sentinel-data/sentinel-2-c1-l2a/22/J/DM/2024/5/S2A_T22JDM_20240508T132235_L2A/S2A_T22JDM_20240508T132235_L2A.json",
    "type": "application/json"
  },
  {
    "rel": "via",
    "href": "s3://sentinel-s2-l2a/tiles/22/J/DM/2024/5/8/0/metadata.xml",
    "type": "application/xml",
    "title": "Granule Metadata in Sinergize RODA Archive"
  },
  {
    "rel": "parent",
    "type": "application/json",
    "href": "https://earth-search.aws.element84.com/v1/collections/sentinel-2-c1-l2a"
  },
  {
    "rel": "collection",
    "type": "application/json",
    "href": "https://earth-search.aws.element84.com/v1/collections/sentinel-2-c1-l2a"
  },
  {
    "rel": "root",
    "type": "application/json",
    "href": "https://earth-search.aws.element84.com/v1"
  },
```

Item Properties


```
"id": "S2A_T22JDM_20240508T132235_L2A",
"properties": {
  "title": "Sentinel-2 Scene",
  "datetime": "2024-05-08T13:30:53.463000Z",
  "created": "2024-05-08T22:20:35.704Z",
  "updated": "2024-05-08T22:20:35.704Z",
  "platform": "sentinel-2a",
  "constellation": "sentinel-2",
  "instruments": ["msi"]
},
"geometry": {
  "type": "Polygon",
```

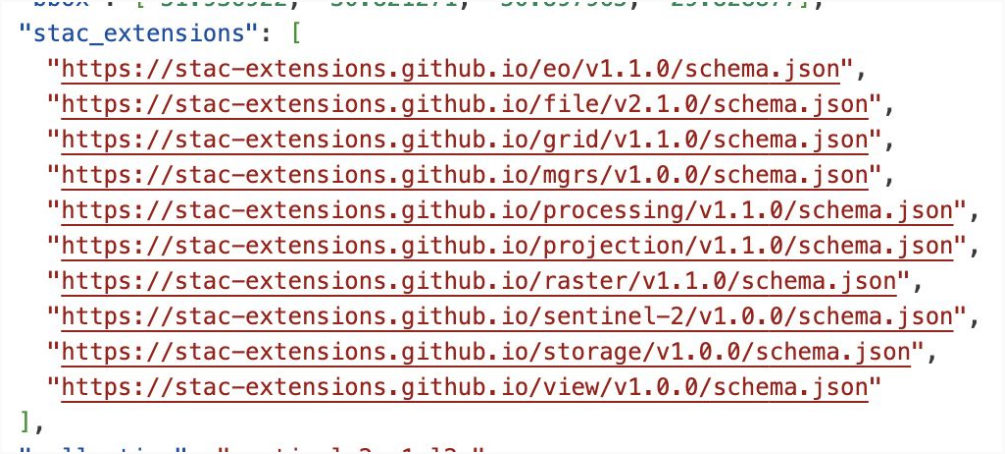
Assets


```
"assets": {
  "red": {
    "href": "https://e84-earth-search-sentinel-data.s3.us-west-2.amazonaws.com/sentinel-2-c1-l2a/22/J/DM/2024/5/S2A_T22JDM_20240508T132235_L2A/B04.tif",
    "type": "image/tiff; application=geotiff; profile=cloud-optimized",
    "title": "Red - 10m",
    "name": "B04",
    "nodata": 0,
    "data_type": "uint16",
    "gsd": 10,
    "roles": ["data", "reflectance"]
  },
  "green": {
    "href": "https://e84-earth-search-sentinel-data.s3.us-west-2.amazonaws.com/sentinel-2-c1-l2a/22/J/DM/2024/5/S2A_T22JDM_20240508T132235_L2A/B03.tif",
    "type": "image/tiff; application=geotiff; profile=cloud-optimized",
    "title": "Green - 10m",
    "name": "B03",
    "nodata": 0,
    "data_type": "uint16",
    "gsd": 10,
    "roles": ["data", "reflectance"]
  },
  "blue": {
```

Bands


```
"visual": {
  "href": "https://e84-earth-search-sentinel-data.s3.us-west-2.amazonaws.com/sentinel-2-c1-l2a/2
  "type": "image/tiff; application=geotiff; profile=cloud-optimized",
  "title": "True color image",
  "bands": [
    {
      "name": "B04",
      "eo:common_name": "red",
      "eo:center_wavelength": 0.665,
      "eo:full_width_half_max": 0.038,
      "nodata": 0,
      "data_type": "uint8",
      "raster:spatial_resolution": 10
    },
    {
      "name": "B03",
      "eo:common_name": "green",
      "eo:center_wavelength": 0.56,
      "eo:full_width_half_max": 0.045,
      "nodata": 0,
      "data_type": "uint8",
      "raster:spatial_resolution": 10
    },
    {
      "name": "B02",
      "eo:common_name": "blue",
      "eo:center_wavelength": 0.49,
      "eo:full_width_half_max": 0.098,
      "nodata": 0,
      "data_type": "uint8",
      "raster:spatial_resolution": 10
    }
  ],
  "gsd": 10,
  "proj:shape": [10980, 10980],
```

What about Extensions?



Extensions?


```
"id": "S2A_T22JDM_20240508T132235_L2A",
"properties": {
  "title": "Sentinel-2 Scene",
  "datetime": "2024-05-08T13:30:53.463000Z",
  "created": "2024-05-08T22:20:35.704Z",
  "updated": "2024-05-08T22:20:35.704Z",
  "platform": "sentinel-2a",
  "constellation": "sentinel-2",
  "instruments": ["msi"],
  "eo:cloud_cover": 72.339511,
  "proj:epsg": 32722,
  "proj:centroid": { "lat": -30.35119, "lon": -51.34972 },
  "mgrs:utm_zone": 22,
  "mgrs:latitude_band": "J",
  "mgrs:grid_square": "DM",
  "grid:code": "MGRS-22JDM",
  "view:azimuth": 102.132995978735,
  "view:incidence_angle": 8.43961873489662,
  "view:sun_azimuth": 33.1498997205534,
  "view:sun_elevation": 35.3222396194671,
  "storage:platform": "AWS",
  "storage:region": "us-west-2",
  "storage:requester_pays": false,
  "processing:software": { "sentinel-2-c1-l2a-to-stac": "v2024.02.01" }
},
"geometry": {
```


EO extension

Projection extension

Mgrs extension

Grid extension

View extension

Storage extension

Processing extension

Common Metadata

https://github.com/radiantearth/stac-spec/blob/master/commons/common-metadata.md

As a user, what software should I use?

pystac-client



A Python client for working with STAC APIs.

Installation

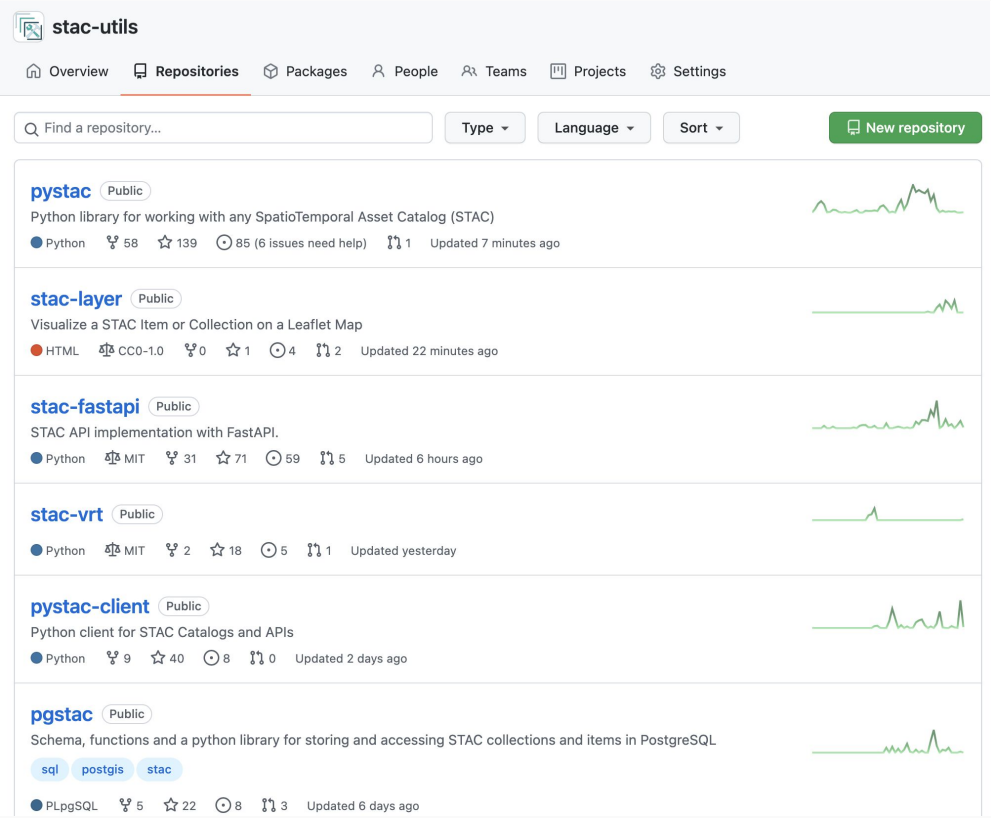
PySTAC Client is published to PyPi as pystac-client.

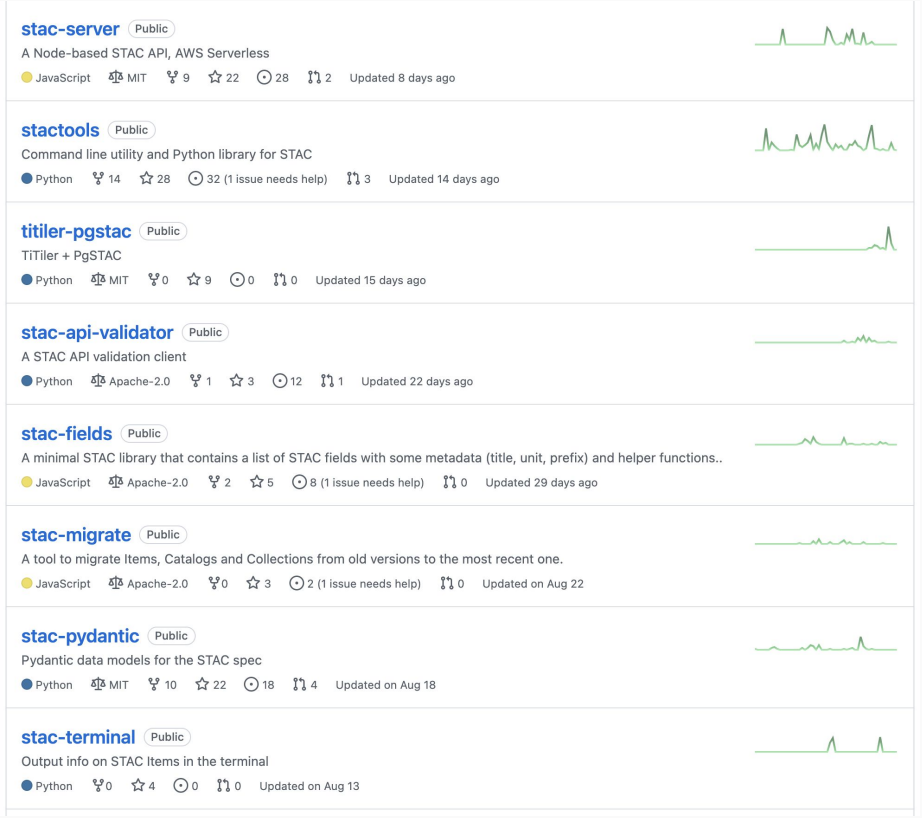
The only direct Python dependencies of **pystac-client** are PySTAC, requests, and dateutil.

To install with pip, run:

```
python -m pip install pystac-client
```

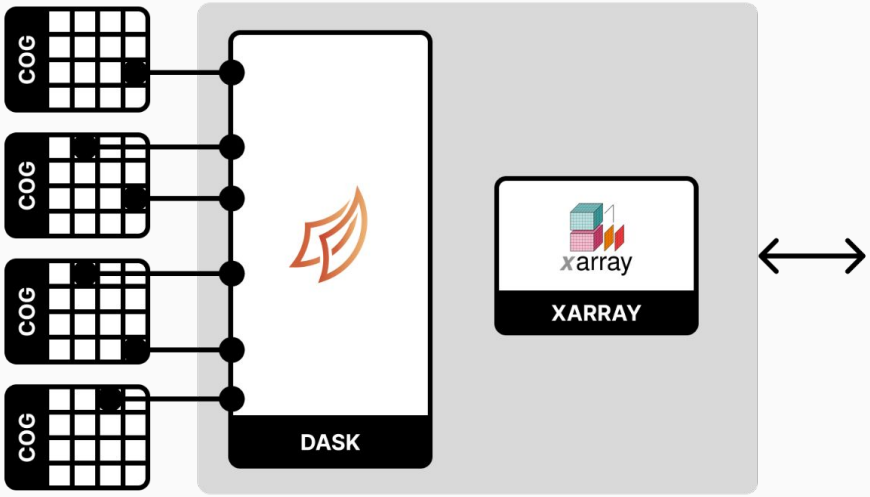
stac-utils


stac-utils
Overview
Repositories
Packages
People
Teams
Projects
Settings
Find a repository...
Type
Language
Sort
New repository
pystac Public
Python library for working with any SpatioTemporal Asset Catalog (STAC)
Python 58 139 85 (6 issues need help) 1 Updated 7 minutes ago
stac-layer Public
Visualize a STAC Item or Collection on a Leaflet Map
HTML CC0-1.0 0 1 4 2 Updated 22 minutes ago
stac-fastapi Public
STAC API implementation with FastAPI.
Python MIT 31 71 59 5 Updated 6 hours ago
stac-vrt Public
Python MIT 2 18 5 1 Updated yesterday
pystac-client Public
Python client for STAC Catalogs and APIs
Python 9 40 8 0 Updated 2 days ago
pgstac Public
Schema, functions and a python library for storing and accessing STAC collections and items in PostgreSQL
sql postgis stac
PLpgSQL 5 22 8 3 Updated 6 days ago


stac-server Public
A Node-based STAC API, AWS Serverless
JavaScript MIT 9 22 28 2 Updated 8 days ago
stactools Public
Command line utility and Python library for STAC
Python 14 28 32 (1 issue needs help) 3 Updated 14 days ago
titiler-pgstac Public
TiTiler + PgSTAC
Python MIT 0 9 0 0 Updated 15 days ago
stac-api-validator Public
A STAC API validation client
Python Apache-2.0 1 3 12 1 Updated 22 days ago
stac-fields Public
A minimal STAC library that contains a list of STAC fields with some metadata (title, unit, prefix) and helper functions..
JavaScript Apache-2.0 2 5 8 (1 issue needs help) 0 Updated 29 days ago
stac-migrate Public
A tool to migrate Items, Catalogs and Collections from old versions to the most recent one.
JavaScript Apache-2.0 0 3 2 (1 issue needs help) 0 Updated on Aug 22
stac-pydantic Public
Pydantic data models for the STAC spec
Python MIT 10 22 18 4 Updated on Aug 18
stac-terminal Public
Output info on STAC Items in the terminal
Python 0 4 0 0 Updated on Aug 13

STAC + OpenDataCube

[pystac-client](#) + [odc-stac](#)



Find STAC Items to Load

```
[3]: km2deg = 1.0 / 111
     x, y = (113.887, -25.843)  # Center point of a query
     r = 100 * km2deg

     query = catalog.search(
         collections=["sentinel-s2-l2a-cogs"],
         datetime="2021-09-16",
         limit=10,
         bbox=(x - r, y - r, x + r, y + r),
     )

     items = list(query.get_items())
     print(f"Found: {len(items):d} datasets")
```

```
Found: 9 datasets
```

Construct Dask Dataset

Note that even though there are 9 STAC Items on input, there is only one timeslice on output. This is because of `groupy="solar_day"`. With that setting `stac_load` will place all items that occured on the same day (as adjusted for the timezone) into one image plane.

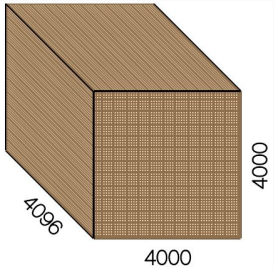
```
[4]: # crs = "epsg:32749"  # native UTM projection in the query region
     crs = "epsg:3857"  # Since we will plot it on a map we need to use 3857 projection
     zoom = 2 ** 5  # overview level 5

     xx = stac_load(
         items,
         output_crs=crs,
         resolution=(-10 * zoom, 10 * zoom),
         chunks={},
         groupby="solar_day",
         measurements=["red", "green", "blue"],
         stac_cfg=cfg,
     )
     xx
```

```
[4]: xarray.Dataset
```

▸ Dimensions: (time: 1, y: 1098, x: 833)

	Array	Chunk
Bytes	524.29 GB	125.96 MB
Shape	(4096, 4000, 4000)	(4096, 62, 62)
Count	258983 Tasks	4225 Chunks
Type	float64	numpy.ndarray



STAC for Providers

STAC Best Practices

https://github.com/radiantearth/stac-spec/blob/master/best-practices.md

STAC Best Practices

> ⚠ Warning
>
> This repository is a very early version and not officially endorsed by the STAC PSC yet. We plan to also move the best practices from the official stac-spec and stac-api-spec repositories over to this repository in the future.
>
> Feedback is always welcome, ideally via GitHub issues or pull requests.

This repository contains the STAC Best Practices.

This repository makes a number of recommendations for creating real world SpatioTemporal Asset Catalogs. None of them are required to meet the core specification, but following these practices will make life easier for client tooling and for users. They come about from practical experience of implementors and introduce a bit more 'constraint' for those who are creating STAC objects representing their data or creating tools to work with STAC.

While the current goal of the core is to remain quite flexible and simple to meet a wide variety of use cases, in time some of these may evolve to become part of the core specification.

STAC Specification

- Web Best Practices
- Item Best Practices
- Asset and Link Best Practices
- Catalog and Collection Best Practices
- Metadata and Extension Best Practices

https://github.com/radiantearth/stac-best-practices/

How should I organize my Collections?

- Consider collections from the user
- Uniform assets



What extensions should I use?

Extension	Minimum fields to include in Item properties
eo	eo:cloud_cover, eo:snow_cover
grid	grid:code
processing	processing:level, processing:datetime, processing:software at a minimum
product	product:type only if using the sar extension
projection	proj:code (if rectified data), proj:centroid
storage	storage:schemes to define type, platform, and region
view	view:off_nadir, view:azimuth and whatever other angles are available
sar	Include any and all fields that are available

What should I use to ‘serve’ STAC?

- None?
  - Static catalog with linked JSON
  - GeoParquet files (STAC geoparquet)
- Which API server should I use
  - stac-fastapi
  - stac-server
  - Pygeoapi
- Backend database?
  - PostGIS
  - OpenSearch/Elastic?

OpenSearch – Pros

👍 Advantage
Very fast for bbox and intersects searches
Scales easily to millions of items
Supports text search and metadata filters
Great for map UIs and faceted browsing

Con: Can be expensive to run at scale; intersections are approximate.

PostGIS – Pros

👍 Advantage
Exact geometry and topology (no approximations)
Supports complex spatial joins and analytics
Good for ingestion QA and coverage analysis
Lower operational cost for smaller workloads

Con: Slower and harder to scale for high-volume searches.

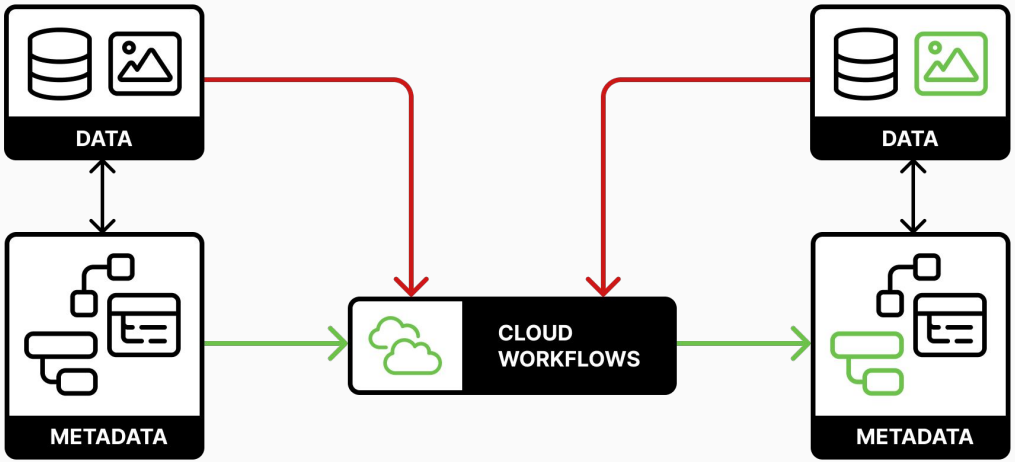
STAC Workflows

Stop thinking of working on files

```
LC08_L1GT_168005_20190828_20190828_01_RT.tif
LC08_L1GT_168005_20190828_20190828_01_RT_B1.TIF
LC08_L1GT_168005_20190828_20190828_01_RT_B3.TIF
LC08_L1GT_168005_20190828_20190828_01_RT_c10_f1.tif
LC08_L1GT_168005_20190828_20190828_01_RT_cm.tif
LC08_L1GT_168005_20190828_20190828_01_RT_k1.tif
LC08_L1GT_168005_20190828_20190828_01_RT_k1_class_c50_i9.tif
LC08_L1GT_168005_20190828_20190828_01_RT_k1_tmp_test5.tif
LC08_L1GT_168005_20190828_20190828_01_RT_k1_ver7.6.tif
LC08_L1GT_168005_20190828_20190828_01_final_class2.tif
LC08_L1GT_168005_20190828_20190828_01_final_version_class.tif
LC08_L1GT_168005_20190828_20190828_01_final_version_final.tif
LC08_L1GT_168005_20190828_20190828_01_test.tif
LC08_L1GT_168005_20190828_20190828_test1.tif
LC08_L1GT_168005_20190828_20190828_test_test.tif
```

Cloud-Native Geospatial Workflows

- Workflows operate on metadata
  - Metadata in, metadata out (STAC In, STAC out)
- Metadata contains complete characterization of data
  - Data location and access



Data provenance


```
"properties": {
  "datetime": "2016-08-22T18:28:23.368922Z",
  "start_datetime": "2016-08-22T18:28:23.368922Z",
  "end_datetime": "2016-08-22T18:28:48.368201Z",
  "created": "2016-08-23T00:38:22Z",
  "platform": "sentinel-1a",
  "constellation": "sentinel-1",
  "mission": "sentinel-1",
  "instruments": [
    "c-sar"
  ],
  "sat:absolute_orbit": 12717,
  "sat:orbit_state": "ascending",
  "sat:relative_orbit": 45,
  "sat:anx_datetime": "2016-08-22T18:24:52.513706Z",
  "sar:instrument_mode": "IW",
  "sar:frequency_band": "C",
  "sar:polarizations": [
    "VV",
    "VH"
  ],
  "sar:product_type": "GRD",
  "processing:lineage": "GRD Post Processing",
  "processing:level": "L1C",
  "processing:facility": "Copernicus S1 Core Ground Segment - DPA",
  "processing:software": {
    "Sentinel-1 IPF": "002.71"
  }
```

```
    "links": [
      {↔},
      {↔},
      {↔},
      {
        "title": "Source STAC Item",
        "rel": "derived_from",
        "href": "https://cirrus-v0-data-1qm7gekzjucbq.s3.us-west-2.amazonaws.com/sentinel-s2-l2a/53/Q/PB/2022/7/S2A_53QPB_20220721_0_L2A/S2A_53QPB_20220721_0_L2A.json",
        "type": "application/json"
      },
      {↔},
      {
        "rel": "parent",
        "href": "https://earth-search.aws.element84.com/v0/collections/sentinel-s2-l2a-cogs"
      },
      {
        "rel": "collection",
        "href": "https://earth-search.aws.element84.com/v0/collections/sentinel-s2-l2a-cogs"
      },
      {
        "rel": "root",
        "href": "https://earth-search.aws.element84.com/v0/"
      }
    ]
  },
```

Challenges

STAC Challenges

- Sheer scale of metadata
- Flexibility is the enemy of interoperability
  - Best practices are often not practiced
  - Proliferation of extensions
- Tendency to shoehorn problems (and metadata) into STAC
  - OGC Features API vs STAC
- Interoperability across providers
  - Differences in metadata
  - Multi-cloud: Authoritative Sources vs Copies
- STAC Governance via steering committee

Introduction to STAC GeoParquet



Radiant Earth · Follow

Published in Radiant Earth Insights · 6 min read · Aug 30, 2024

9 1

By Tom Augspurger, Kyle Barron, and Chris Holmes

This post introduces STAC GeoParquet, a specification and library for storing and serving SpatioTemporal Asset Catalogs (STAC) metadata as GeoParquet. By building on GeoParquet, STAC GeoParquet makes it easy to store, transmit, and analyze large collections of STAC items. It makes for a nice complement to a STAC API.

https://medium.com/radiant-earth-insights/introduction-to-stac-geoparquet-031a7902e134

STAC Profiles from CEOS-ARD

STAC Extensions

This profile doesn't define new STAC fields, it's just a profile that uses existing STAC extensions to map and fulfill the CEOS-ARD requirements.

As the identifier for this profile is just a collection of existing extensions and only defines required fields, you get a good bit of validation already without providing the identifier/schema of this profile in `stac_extensions`. If your metadata is already compliant to this profile, you can omit the identifier for this profile in `stac_extensions` to avoid costly regeneration of the Items. You won't get validation whether all required fields are present, but this could be checked manually in the CEOS assessment/review.

The following STAC extensions are relevant for this profile:

Name	Schema URI for `stac_extensions`	Required
Accuracy	`https://stac-extensions.github.io/accuracy/v1.0.0-beta.1/schema.json`	✕
CEOS-ARD	`https://stac-extensions.github.io/ceos-ard/v0.2.0/optical/schema.json`	✓
Classification	`https://stac-extensions.github.io/classification/v2.0.0/schema.json`	✕
Electro Optical	`https://stac-extensions.github.io/eo/v2.0.0/schema.json`	✓
Processing	`https://stac-extensions.github.io/processing/v1.2.0/schema.json`	✕
Projection	`https://stac-extensions.github.io/projection/v2.0.0/schema.json`	✓
Raster	`https://stac-extensions.github.io/raster/v2.0.0/schema.json`	✓
View Geometry	`https://stac-extensions.github.io/view/v1.1.0/schema.json`	✓

Cloud-Native Geospatial Forum (CNG)
6,386 followers
3w •

✨ Introducing the new **STACspec** Project Steering Committee (PSC) ✨ We've got some fresh faces joining the PSC, and we couldn't be more excited! Say hello to:

- **Alex Leith GAICD** (**Auspatious**)
- **Emmanuel Mathot** (**Development Seed**)
- **Julia Signell** (**Element 84**)
- **Michelle Roby** (**Radiant Earth**)
- **Peter Gadomski** (**Development Seed**)

They'll be joining **Matthias Mohr** (Independent) and **Matthew Hanson** (**Element 84**) in steering STAC's future.

🙏 We'd like to give a huge thank you to **Chris Holmes** (**Planet**), **James B.** (**Sparkgeo - Geospatial Consulting & Development**), and **Rob Emanuele** (**Microsoft**) for everything they've done to make STAC what it is today. You've helped build something incredible, and we're so grateful!









Join the community meetings!



https://groups.google.com/a/cloudnativegeo.org/g/stac-community

Thank you!

Questions?



Matthew Hanson
@geoskeptic.bsky.social

Element 84

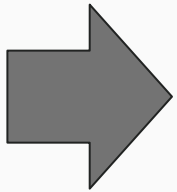


What’s the difference between raster:bands and eo:bands?

Should I be using them both?

STAC 1.1 Bands

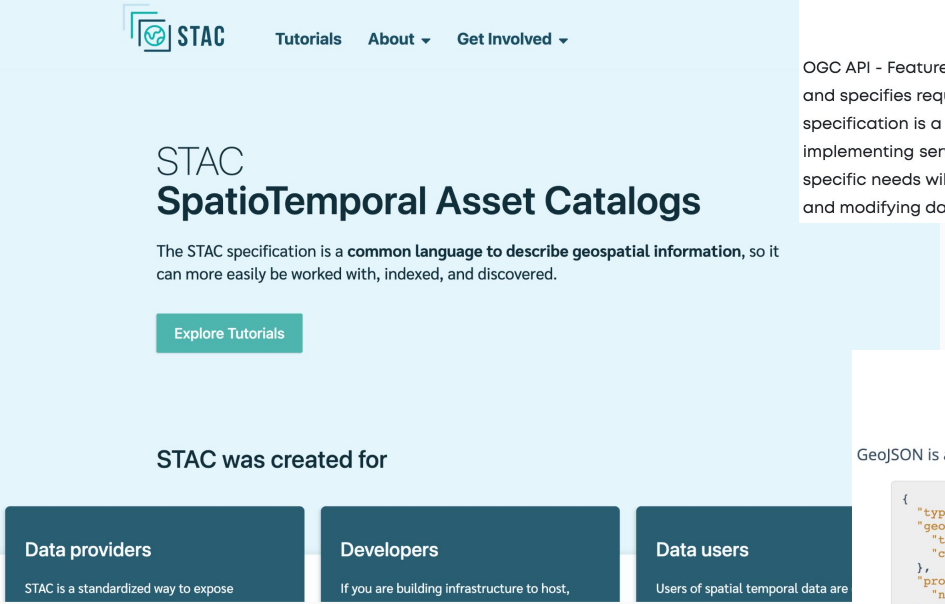
```
{
    "eo:bands": [{
     "name": "B4",
     "description": "Red band",
     "common_name": "red",
     "center_wavelength": 0.665,
     "full_width_half_max": 0.038
    }],
    "raster:bands": [{
     "data_type": "float32",
     "nodata": "nan",
     "statistics":
        {"minimum": 0, "maximum": 1},
     "spatial_resolution": 10
    }]
}
```



```
{
    "bands": [{
        "name": "B4",
        "description": "Red band",
        "data_type": "float32",
        "nodata": "nan",
        "statistics":
            {"minimum": 0, "maximum": 1},
        "eo:common_name": "red",
        "eo:center_wavelength": 0.665,
        "eo:full_width_half_max": 0.038,
        "raster:spatial_resolution": 10
    }]
}
```

Can I use STAC for vector data?

STAC vs OGC API - Features



OGC API - FEATURES

OGC API - Features is a multi-part standard that offers the capability to create, modify, and query spatial data on the Web and specifies requirements and recommendations for APIs that want to follow a standard way of sharing feature data. The specification is a multi-part document. The Core part of the specification describes the mandatory capabilities that every implementing service has to support and is restricted to read-access to spatial data. Additional capabilities that address specific needs will be specified in additional parts. Envisaged future capabilities include, for example, support for creating and modifying data, more complex data models, richer queries, and additional coordinate reference systems.

GeoJSON

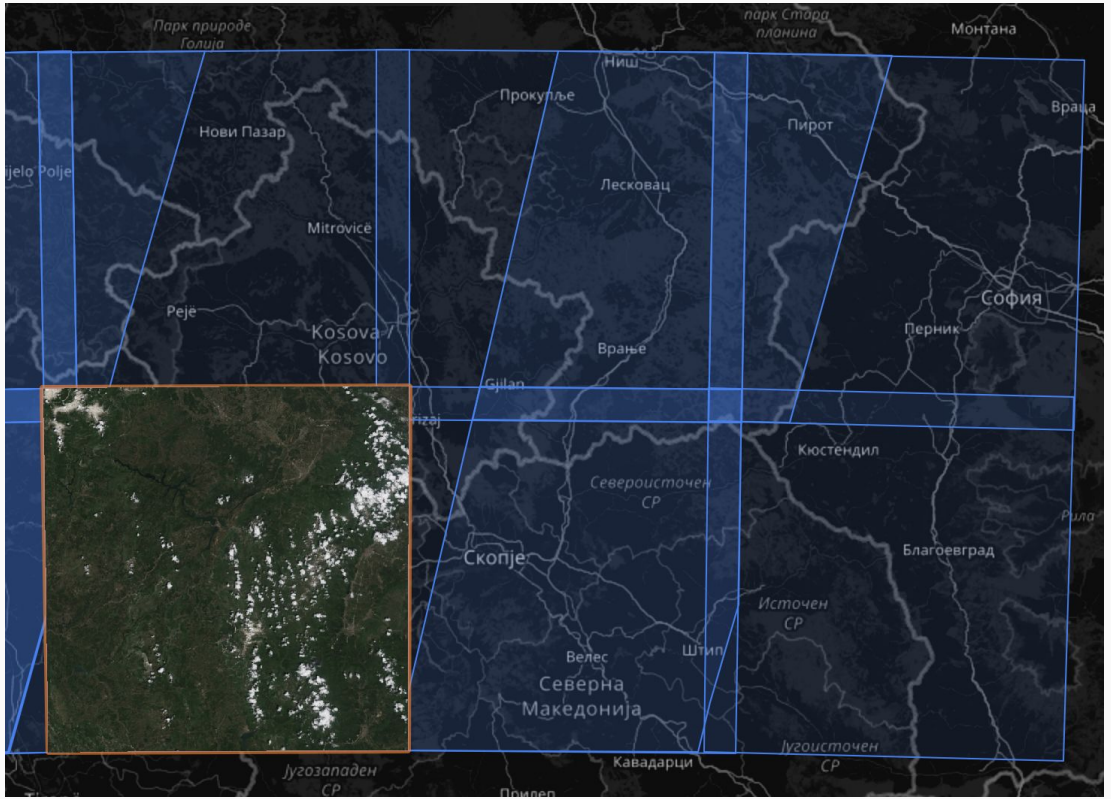
GeoJSON is a format for encoding a variety of geographic data structures.

```
{
  "type": "Feature",
  "geometry": {
    "type": "Point",
    "coordinates": [125.6, 10.1]
  },
  "properties": {
    "name": "Dinagat Islands"
  }
}
```

GeoJSON supports the following geometry types: `Point`, `LineString`, `Polygon`, `MultiPoint`, `MultiLineString`, and `MultiPolygon`. Geometric objects with additional properties are `Feature` objects. Sets of features are contained by `FeatureCollection` objects.

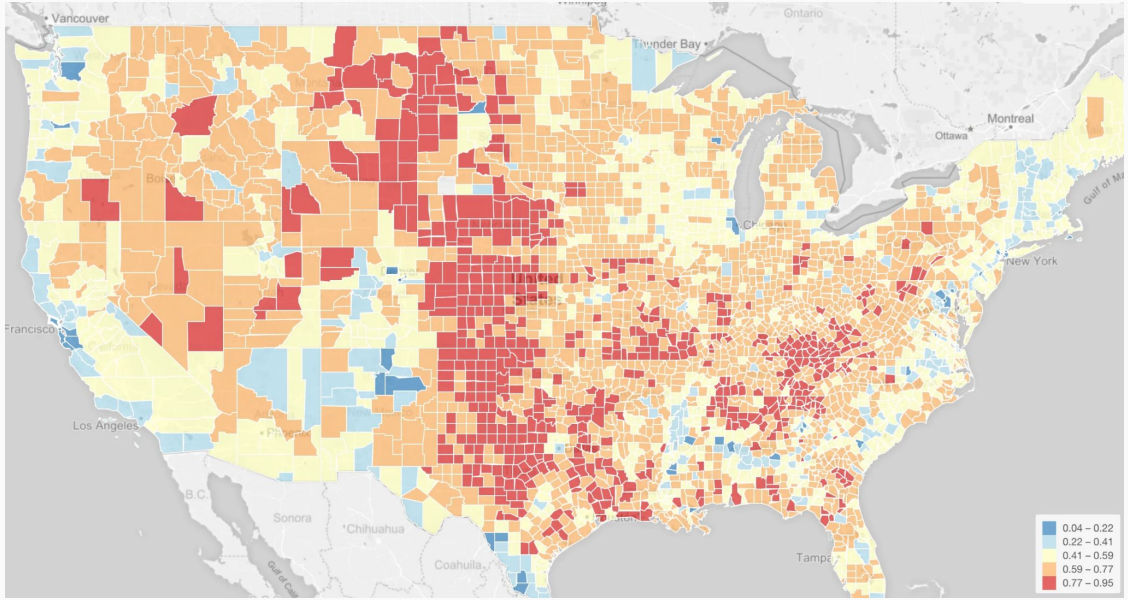
STAC Items

- GeoJSON
- Defines region where there is data
- May contain additional properties
- Data is in separate files (assets)



OGC Features

- GeoJSON
- Defines regions of uniform properties



Use OGC Features if geometry is the data

Use STAC if geometry defines where data is

Google-Microsoft Open Buildings - combined by VIDA

Overview

This dataset merges Google's V3 Open Buildings and Microsoft's latest Building Footprints. With 2,534,595,270 footprints, it is to our knowledge the most complete openly available dataset as of Sep. 2023. It covers 92% of Level 0 administrative boundaries, and is divided into 182 partitions. Each footprint is labelled with its respective source, either Google or Microsoft. It can be accessed in cloud-native geospatial formats such as GeoParquet, FlatGeobuf and PMTiles.

See it in action

You can Observable to get a quick overview of the dataset or go to VIDA to see it in action.

Original datasets

Google

The original Google V3 open buildings is downloadable from this link as gzipped CSV files. Here are some key details about the original dataset:

> *The dataset contains 1.8 billion building detections, across an inference area of 58M km2 within Africa, South Asia, South-East Asia, Latin America and the Caribbean.*
>
> *Each building in the dataset has a polygon defining its footprint on the ground, a confidence score indicating how certain we are that this is a building, and a Plus Code corresponding to the centre of the building. There is no information about the type of building, its street address, or any details other than its geometry.*

For more comprehensive information, please visit the description page. You can also check out the FAQ section for additional information.

Microsoft

The latest version of Microsoft's building footprints can be downloaded from Microsoft Planetary Computer as gzipped partitioned files.

> *The Microsoft Global Open Buildings dataset was generated through Bing Maps, which detected a total of 1.24 billion buildings. These buildings were identified using imagery from Bing Maps, encompassing data collected between 2014 and 2023, including images from Maxar, Airbus, and IGN France.*

For more detailed information, please visit the github page.

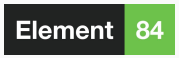
STAC for partitioned vector data

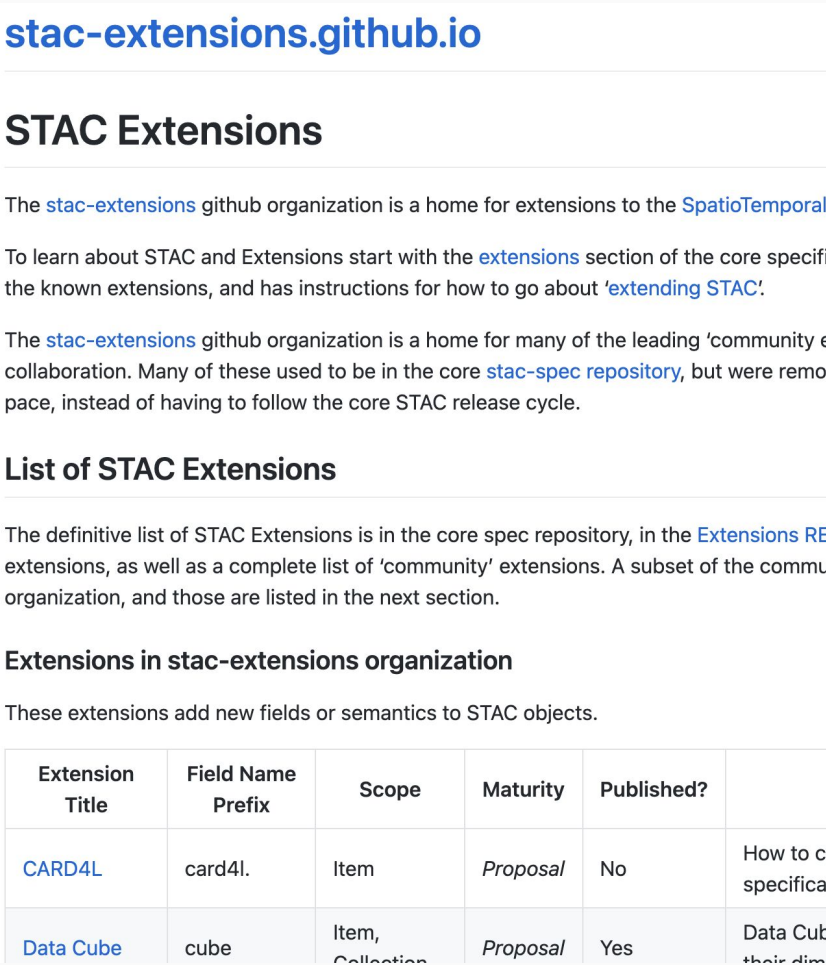
- Very large datasets
  - Avoid expense and complexity of scaling Features API
- Data provider/publisher
  - Focus on creating and distribution, not usage
- Archival
  - Publishing dataset for reproducibility (e.g., scientific study)
- Data Formats
  - FlatGeoBuf
  - GeoParquet
  - PMTiles

There’s an awful lot of extensions, should I look at all of them?

Extensions

- 78 Extensions in the definitive list
  - https://stac-extensions.github.io/
- 66 hosted in GitHub organization
  - https://github.com/stac-extensions
- Extensions rated by “Maturity”
  - Stable
  - Candidate
  - Pilot
  - Proposal
  - Deprecated





stac-extensions.github.io

STAC Extensions

The stac-extensions github organization is a home for extensions to the SpatioTemporal

To learn about STAC and Extensions start with the extensions section of the core specifi
the known extensions, and has instructions for how to go about ‘extending STAC’.

The stac-extensions github organization is a home for many of the leading ‘community e
collaboration. Many of these used to be in the core stac-spec repository, but were remo
pace, instead of having to follow the core STAC release cycle.

List of STAC Extensions

The definitive list of STAC Extensions is in the core spec repository, in the Extensions RE
extensions, as well as a complete list of ‘community’ extensions. A subset of the commu
organization, and those are listed in the next section.

Extensions in stac-extensions organization

These extensions add new fields or semantics to STAC objects.

Extension Title	Field Name Prefix	Scope	Maturity	Published?	
CARD4L	card4l.	Item	*Proposal*	No	How to c specifica
Data Cube	cube	Item, Collection	*Proposal*	Yes	Data Cub their dim

Extensions by Maturity

Stable

- Electro-Optical
- File Info
- Landsat
- Projection
- Scientific Citation
- View Geometry

Candidate

- Datacube
- Processing
- Raster
- SAR
- Satellite
- Sentinel-2
- Versioning Indicators

Deprecated

- Deep Learning Model*
- Item Assets Definition
- Sentinel-3
- Single File STAC
- Time Series

Pilot Extensions

- Alternate Assets
- Anonymized Location
- CARD4L (Optical and SAR)
- Classification
- Grid
- Label
- Machine Learning Model Extension*
- Military Grid Reference System
- NOAA Geostationary Operational Environmental Satellite (GOES)
- NOAA MRMS QPE
- Order
- Point Cloud
- Stats
- Storage
- Table
- Timestamps
- xarray Assets

Proposal Extensions

- Accuracy
- Altimetry
- Attribution
- Authentication
- CEOS-ARD
- CF
- Contacts
- Disasters Charter*
- Example Links
- Forecast
- InSAR
- Language (I18N)
- ML AOI
- ML Model
- Open Science Catalog
- Perspective Imagery
- Product
- Quality*
- Rendering
- Sentinel-1
- Sentinel-5P
- Solar System
- Stereo Imagery
- Themes
- Tiled Assets
- TrainingDML-AI*
- U.S. Fish & Wildlife Service (FWS) National Wetlands Inventory (NWI)
- Video
- Virtual Assets
- Web Map Links

WIP Extensions

- Aerial Photography*
- Archive
- CMIP6
- Camera*
- Composite
- Earthquake
- Film*
- Hyperspectral Imagery
- Scanning*

“Content” Extensions

Electro-Optical

Field Name	Type	Description
eo:bands	[Band Object]	An array of available bands where each object is a Band Object. If given, requires at least one band.
eo:cloud_cover	number	Estimate of cloud cover

SAR

Field Name	Type	Description
sar:instrument_mode	string	**REQUIRED.** The name of the sensor acquisition mode that is commonly used. This should be the short name, if available. For example, `WV` for "Wave mode" of Sentinel-1 and Envisat ASAR satellites.
sar:frequency_band	string	**REQUIRED.** The common name for the frequency band to make it easier to search for bands across instruments. See section "Common Frequency Band Names" for a list of accepted names.
sar:center_frequency	number	The center frequency of the instrument, in gigahertz (GHz).
sar:polarizations	[string]	**REQUIRED.** Any combination of polarizations.
sar:product_type	string	**REQUIRED.** The product type, for example `SSC`, `MGD`, or `SGC`
sar:resolution_range	number	The range resolution, which is the maximum ability to distinguish two adjacent targets perpendicular to the flight path, in meters (m).
sar:resolution_azimuth	number	The azimuth resolution, which is the maximum ability to distinguish two adjacent targets parallel to the flight path, in meters (m).
sar:pixel_spacing_range	number	The range pixel spacing, which is the distance between adjacent pixels perpendicular to the flight path, in meters (m). Strongly RECOMMENDED to be specified for products of type `GRD`.
sar:pixel_spacing_azimuth	number	The azimuth pixel spacing, which is the distance between adjacent pixels parallel to the flight path, in meters (m). Strongly

View

Field Name	Type	Description
view:off_nadir	number	The angle from the sensor between nadir (straight down) and the scene center. Measured in degrees (0-90).
view:incidence_angle	number	The incidence angle is the angle between the vertical (normal) to the intercepting surface and the line of sight back to the satellite at the scene center. Measured in degrees (0-90).
view:azimuth	number	Viewing azimuth angle. The angle measured from the sub-satellite point (point on the ground below the platform) between the scene center and true north. Measured clockwise from north in degrees (0-360).
view:sun_azimuth	number	Sun azimuth angle. From the scene center point on the ground, this is the angle between truth north and the sun. Measured clockwise in degrees (0-360).
view:sun_elevation	number	Sun elevation angle. The angle from the tangent of the scene center point to the sun. Measured from the horizon in degrees (`-90` - `90`). Negative values indicate the sun is below the horizon, e.g. sun elevation of -10° means the data was captured during nautical twilight.

“Usage” Extensions

projection

Field Name	Type	Description
proj:epsg	integer|null	**REQUIRED.** EPSG code of the datasource
proj:wkt2	string|null	WKT2 string representing the Coordinate Reference System (CRS) that the `proj:geometry` and `proj:bbox` fields represent
proj:projjson	PROJJSON Object|null	PROJJSON object representing the Coordinate Reference System (CRS) that the `proj:geometry` and `proj:bbox` fields represent
proj:geometry	GeoJSON Geometry Object	Defines the footprint of this Item.
proj:bbox	[number]	Bounding box of the Item in the asset CRS in 2 or 3 dimensions.
proj:centroid	Centroid Object	Coordinates representing the centroid of the Item (in lat/long)
proj:shape	[integer]	Number of pixels in Y and X directions for the default grid
proj:transform	[number]	The affine transformation coefficients for the default grid

Raster (v2)

Field Name	Type	Description
raster:nodata	number|string	Pixel values used to identify pixels that are nodata in the band either by the pixel value as a number or `nan`, `inf` or `-inf` (all strings).
raster:sampling	string	One of `area` or `point`. Indicates whether a pixel value should be assumed to represent a sampling over the region of the pixel or a point sample at the center of the pixel.
raster:data_type	string	The data type of the pixels in the band. One of the data types as described below.
raster:bits_per_sample	number	The actual number of bits used for this band. Normally only present when the number of bits is non-standard for the `datatype`, such as when a 1 bit TIFF is represented as byte.
raster:spatial_resolution	number	Average spatial resolution (in meters) of the pixels in the band.
raster:statistics	Statistics Object	Statistics of all the pixels in the band.
raster:unit	string	Unit denomination of the pixel value.
raster:scale	number	Multiplicator factor of the pixel value to transform into the value (i.e. translate digital number to reflectance).
raster:offset	number	Number to be added to the pixel value (after scaling) to transform into the value (i.e. translate digital number to reflectance).
raster:histogram	Histogram Object	Histogram distribution information of the pixels values in the band.

storage

Field Name	Type	Description
storage:platform	string	The cloud provider where data is stored
storage:region	string	The region where the data is stored. Relevant to speed of access and inter region egress costs (as defined by PaaS provider)
storage:requester_pays	boolean	Is the data requester pays or is it data manager/cloud provider pays. *Defaults to false*
storage:tier	string	The title for the tier type (as defined by PaaS provider)

Thank you!

Any more questions?



Matthew Hanson
@geoskeptic.bsky.social

Element 84









